Szakdolgozat

Miskolci Egyetem

Szkriptelési megoldások a játékfejlesztésben

Készítette:

Lengyel Gábor

Programtervező informatikus

Témavezető:

Dr. Mileff Péter

Miskolc, 2026

Miskolci Egyetem
Gépészmérnöki és Informatikai Kar
Alkalmazott Matematikai Intézeti Tanszék **Szám:**

Szakdolgozat Feladat

Lengyel Gábor (GKIU70) programtervező informatikus jelölt részére.

A szakdolgozat tárgyköre: Számítógépes játékfejlesztés

A szakdolgozat címe: Szkriptelési megoldások a játékfejlesztésben

A feladat részletezése:

A hallgató feladata, hogy áttekintse a játékfejlesztésben jelenleg elérhető legismertebb szkriptelési technikákat és megoldásokat. Mutasson be néhány olyan létező játékot, amely ilyen technikán alapul. Tervezzen olyan saját játékmotort, amely valamilyen modern szkriptelési (akár egyedi) megoldásra épül, lehetővé teszi a játék bizonyos funkcióinak konfigurálását, szkriptelését, vezérlését. Implementálja az alkalmazást, majd részletesen mutassa be a megvalósítás lépéseit és az elért eredményeket.

Témavezető: Dr. Mileff Péter, egyetemi docens

A feladat kiadásának ideje: 2024. szeptember 20.

. . . . . .
szakfelelős

Eredetiségi Nyilatkozat

Alulírott **Lengyel Gábor**; Neptun-kód: `GKIU70` a Miskolci Egyetem Gépészmérnöki és Informatikai Karának végzős Programtervező informatikus szakos hallgatója ezennel büntetőjogi és fegyelmi felelősségem tudatában nyilatkozom és aláírásommal igazolom, hogy *Szkriptelési megoldások a játékfejlesztésben* című szakdolgozatom saját, önálló munkám; az abban hivatkozott szakirodalom felhasználása a forráskezelés szabályai szerint történt.

Tudomásul veszem, hogy szakdolgozat esetén plágiumnak számít:

- szószerinti idézet közlése idézőjel és hivatkozás megjelölése nélkül;
- tartalmi idézet hivatkozás megjelölése nélkül;
- más publikált gondolatainak saját gondolatként való feltüntetése.

Alulírott kijelentem, hogy a plágium fogalmát megismertem, és tudomásul veszem, hogy plágium esetén szakdolgozatom visszautasításra kerül.

Miskolc, ... év ..... hó ..... nap

.....

Hallgató

1. A szakdolgozat feladat módosítása

szükséges (módosítás külön lapon)
nem szükséges

....
dátum témavezető(k)

2. A feladat kidolgozását ellenőriztem:

témavezető (dátum, aláírás):
.....
.....
.....

konzulens (dátum, aláírás):
.....
.....
.....

3. A szakdolgozat beadható:

....
dátum témavezető(k)

4. A szakdolgozat ..... szövegoldalt
..... program protokollt (listát, felhasználói leírást)
..... elektronikus adathordozót (részletezve)
.....
..... egyéb mellékletet (részletezve)
.....
tartalmaz.

....
dátum témavezető(k)

5. A szakdolgozat bírálatra

bocsátható
nem bocsátható

A bíráló neve: .....

....
dátum szakfelelős

6. A szakdolgozat osztályzata

a témavezető javaslata: .....
a bíráló javaslata: .....
a szakdolgozat végleges eredménye: .....

Miskolc, ...
a Záróvizsga Bizottság Elnöke

Tartalomjegyzék

1. Bevezetés **1**

2. A videójátékok története **2**

3. A játékfejlesztés folyamata **5**
3.1. Szoftverfejlesztési életciklus 5
3.1.1. Követelmények meghatározása 5
3.1.2. Tervezési fázis 6
3.1.3. Pre-produkciós fázis 6
3.1.4. Gyártás 7
3.1.5. Tesztelés 7
3.1.6. Karbantartás 8
3.2. A játékmotor 8
3.2.1. Unreal Engine 9
3.2.2. Unity 9
3.2.3. Godot 9
3.2.4. Egyedi játékmotorok 9

4. A szkriptelés szerepe a játékfejlesztésben **11**
4.1. Mi a szkriptelés? 11
4.2. A programozás és a szkriptelés összehasonlítása 11
4.3. A szkriptelés előnyei 12
4.4. A szkriptelés korlátai 13
4.5. Szkriptnyelvek 13
4.5.1. JavaScript 13
4.5.2. Lua 13
4.5.3. Ruby 14
4.5.4. Egyedi szkriptnyelvek 14
4.6. Szkriptelési megoldások 14
4.6.1. Végrehajtási mechanizmusok 14
4.6.2. A szkript és a játékmotor kapcsolata 15
4.7. Szkriptelhető játékelemek 16
4.7.1. Játékmenet 16
4.7.2. HUD 16
4.8. Példák szkriptelést alkalmazó játékokra 16
4.8.1. Angry Birds 16
4.8.2. Don't Starve 16
4.8.3. World of Warcraft 16
4.8.4. Roblox 16

4.8.5. The Sims 4 16
4.8.6. The Elder Scrolls: Skyrim 16
4.8.7. Grand Theft Auto játékok 16

5. Fejlesztési dokumentáció 17
5.1. Követelmények meghatározása 17
5.1.1. Funkcionális követelmények 17
5.1.2. Nem-funkcionális követelmények 18
5.2. Tervezés 19
5.3. A fejlesztőkörnyezet bemutatása 19
5.4. A játékmotor megírása 19
5.4.1. Ablakkezelés 19
5.4.2. A színtér 19
5.4.3. Kamerakezelés és irányítás 19
5.4.4. Objektum- és textúrakezelés 19
5.4.5. Játékfizika 19
5.4.6. Ütközésérzékelés 19
5.4.7. Kétdimenziós grafika 19
5.4.8. Hangok 19
5.5. A szkriptelési rendszer implementálása 19
5.6. A játékmenet kialakítása 19
5.7. A felhasználói felület 19
5.7.1. Menü 19
5.7.2. HUD 19
5.8. Tesztelés 19
5.8.1. Funkcionális tesztelés 19
5.8.2. Nem-funkcionális tesztelés 19

6. Összefoglalás 20

1. fejezet

Bevezetés

A játékfejlesztés kétségtelenül egy jelentős, nagy volumenű területe a szoftverfejlesztésnek. Ez egy rendkívül komplex ága az informatikának, amely olyan informatikai szakterületeket ölel fel, mint a számítógépi grafika vagy a mesterséges intelligencia. Emellett számos művészeti és kreatív elem is kiemelt szerepet játszik egy videojáték elkészítésében. Központi szerepet kaphat a történetmesélés, a zeneszerzés, sőt még a színészi munka is. Éppen ezért egy videójáték létrehozása legalább annyira művészet, mint egy festmény megfestése, egy dal megkomponálása vagy egy film elkészítése.

Az elmúlt évtizedekben a játékszoftverek rohamos fejlődésen mentek keresztül. Míg a kezdeti játékok egyszerű grafikával és korlátozott funkcionalitással rendelkeztek, a mai játékok már a valósághoz közeli vizuális élményeket és gazdag, sokrétű játékmenetet kínálnak. A technológia fejlődése lehetővé tette az élethű fizikai szimulációkat és interaktív világok részletgazdag megalkotását. A játékfejlesztésben a különböző szkriptelési megoldások nemcsak a játék gyorsabb és egyszerűbb fejlesztését teszik lehetővé, hanem növelik a testreszabhatóságot és a rugalmasságot, mivel a játék bizonyos részei dinamikusan kezelhetők és konfigurálhatók. A legfontosabb, hogy mindez úgy valósítható meg, hogy nem szükséges mélyen belenyúlni a játékmotorba, de még csak ismerni sem feltétlenül szükséges.

Ma is számos multiplayer játék létezik, amit a szkriptelési lehetőségek tartanak a köztudatban. A játékosok létrehozhatnak, vagy csatlakozhatnak más játékosok által létrehozott és fejlesztett játékszerverekhez, amiknek a műfaja és minősége gyakran csak a fejlesztők kreativitásán és munkáján múlik. A szkriptelési folyamatokat kisebb-nagyobb részletességű dokumentációk segítik, és alapszintű programozási ismeretekkel könnyen el lehet készíteni egyszerűbb, de izgalmas játékmódokat. Az elmúlt években jelentős időt töltöttem el ilyen játékokkal játékosként és fejlesztőként is, ezért döntöttem úgy, hogy elmélyítem a tudásomat a szkriptelésben.

A szakdolgozatom célja, hogy bemutasson különféle szkriptelési megoldásokat a játékfejlesztésben. A dolgozat keretein belül egy saját játékmotor fejlesztésére is sor kerül, mely szkriptelési lehetőségeket nyújt egy háromdimenziós platformer játék elkészítéséhez. Továbbá bemutatásra kerül a tervezés és az implementáció folyamata is.

2. fejezet

A videójátékok története

A videójátékok a szórakozás egy formája, amely valamilyen elektronikus eszköz – számítógép, konzol, mobiltelefon vagy egyéb hardver - használatával történik. Ahogy az informatika és később az internet is egyre inkább átalakította a világot és jelentős mértékben felgyorsította, valamint új lehetőségekkel bővítette ki az életünket, úgy a videójátékok is új kikapcsolódási lehetőségeket kínálnak számtalan formában és műfajban.

A videójátékok évről évre egyre népszerűbbek, és mára egy hatalmas iparággá nőtték ki magukat, amely minden nem és korosztály számára szórakozási lehetőséget biztosít. Aki nyitott a videójátékra, biztosan találhat olyan stílusú játékot, amiben szívesen elmerül: elérhetők számítógépes, játékkonzolos, mobiltelefonos játékok is a történelmi, stratégiai játékoktól kezdve az interaktív, filmszerű és történetközpontú játékokon át az alkalmi, ügyességi játékokig.

A videójátékok története egészen a 20. század közepéig nyúlik vissza. Ekkor alkották meg az első videójátékokat, amik persze még meglehetősen kezdetlegesek voltak mai szemmel nézve. Ezek ekkor még inkább tudományos kísérletek és bemutatók voltak, nem pedig széles körben elérhető szórakoztató termékek. Az 1958-os „Tennis for Two" például analóg számítógépre készült tenisz szimulátor volt, grafikus megjelenítésére pedig egy oszcilloszkópot használt. Ez egy jelentős mérföldkő volt, hiszen ez volt az első olyan játék, ami grafikus kijelzőt használt. Szintén jelentős lépés volt a „Spacewar!" nevű játék, amely 1962-ben készült és egy PDP-1 számítógépen üzemelt. Annak ellenére, hogy akkoriban ezek a játékok még kevés emberhez jutottak el, jelentős inspirációt jelentettek a későbbi fejlesztések számára.

Az 1970-es években jelentős áttörés történt: ekkor jelentek meg az első játéktermi játékok, például a sokak által jól ismert „Pong", amelyet 1972-ben dobtak piacra és hatalmas népszerűségre tett szert. Ez a játék is teniszt szimulált, amelyben két játékosnak kellett egymásnak ütögetnie a labdát. A játékok ekkor még rendkívül egyszerűek voltak, kezdetben még csak néhány pixeles alakzatokat vagy karaktereket jelenítettek meg, azonban ezeknek a játékoknak a sikere és népszerűsége utat adott a játék-ipar további fejlődésének.

Az 1980-as években a játékkonzolok és a számítógépek elterjedésével lehetőség nyílt az otthoni videójátékozásra is. Az Atari 2600 volt az egyik első otthoni konzol, amely széles körben elérhetővé vált. Az 1980-as évek végére megjelent a Nintendo Entertainment System konzol. Olyan korai személyi számítógépek is megjelentek az évtized során, mint a Commodore 64, ami szintén meghatározó szerepet játszott. Ebben az időszakban olyan ikonikus játékok jelentek meg, mint a „Pac-Man" vagy a „Super Ma-

rio Bros.”. Ezeknek a játékoknak már részletesebb, látványosabb volt a megjelenésük, de még mindig hosszú fejlődés várt rájuk.

Az 1990-es évek hozták meg az igazi áttörést a videójátékok grafikájának terén. Ebben az évtizedben jelentek meg az első háromdimenziós játékok, amik forradalmasították a videójátékok megjelenését. Az egyik legjelentősebb játék ebből az időszakból az 1993-ban kiadott „Doom”, ami egy belső nézetű lövöldözős (First Person Shooter) játék, de ezekben az években jelent meg a „Super Mario 64” is, ami az első háromdimenziós platformjátékok egyike volt. Ezek a műfajok napjainkban is nagy népszerűségnek örvendenek.

A 2000-es években a játékok grafikája, fizikája és animációi egyre valósághűbbek lettek. Az új renderelési technikák még részletgazdagabb megjelenést tettek lehetővé, a fejlett fizikai motorok pedig előtte még nem látott szinten szimulálták a világunkat. Olyan kultusz játékok jelentek meg a 2000-es évek elején, mint a „Grand Theft Auto: San Andreas” és a „Half-Life 2”, amelyekkel még ma is szívesen játszanak a játékosok.

Az internet, ahogy a világon rengeteg más területet, úgy a videójátékokat is forradalmasította és új lehetőségeket nyitott meg. Míg az internet elterjedése előtt a játékokat csak egyedül vagy együtt, fizikailag egy helyen tartózkodva lehetett játszani, az internet megjelenésével teret kaptak az online videójátékok, ami így már lehetővé tette, hogy akár a barátainkkal, akár idegenekkel játsszunk együtt a világ bármely pontjáról. Ebben az időszakban jelent meg az MMORPG műfaj is, amely hatalmas virtuális világokat kínált, ahol a játékosok egymással interakcióba léphettek és közösen fejlődhettek, és ezáltal nagy közösségek jöttek létre. Az egyik legismertebb MMORPG a 2004-es megjelenésű „World of Warcraft”, ami sok millió játékost csábított el.

A mobiltelefonok fejlődése, az okostelefonok és a tabletek lehetővé tették, hogy a videójátok szinte bárki számára, bármikor elérhetők legyenek, helyhez kötöttség nélkül. Ez a könnyű hozzáférés még elterjedtebbé és sikeresebbé tette a játékipart, és még inkább beférkőzött a mindennapjainkba. Az olyan sikerjátékok, mint a „Pokémon GO” vagy a „Candy Crush” bebizonyították, hogy a játékok alkalmi felhasználók körében is népszerűek lehetnek.

Napjainkban a játékfejlesztés már egy kiforrott, de ettől függetlenül folyamatosan fejlődő iparág. A mai játékok képesek rendkívül valósághű megjelenést és filmszerű élményt nyújtani. Napjainkban gyakran emlegetett technológia a ray tracing, vagyis a sugárkövetés, amely segítségével sokkal élethűbb, valós idejű fényhatások, tükröződések hozhatók létre a játékokban. Ezt a technológiát korábban is használták már filmek gyártásakor, a videójátékokban azonban csak az elmúlt években honosodott meg. A jelenleg is rohamosan fejlődő mesterséges intelligencia a játékfejlesztés sok pontján alkalmazható, és ma már egyre inkább alkalmazzák is. A nem játékos által irányított karakterek viselkedése, a procedurális tartalomgenerálás, de akár a hangfelismerés és az ez alapján történő interaktív párbeszédek is jó példák arra, hogy milyen formában lehet jelen a mesterséges intelligencia egy videójátékban.

A virtuális valóság is egy teljesen új fejezetet nyitott a videójátékok történetében. A VR új szintre emelte a játékélményt, lehetővé teszi azt, hogy a játékosok közvetlenül részesei legyenek a játéknak. A VR, illetve a mesterséges intelligencia is fejlődése nagy szerepet játszhat a videójátékok jövőjében.

A videójátékok ma már nem csupán szórakozási lehetőséget jelentenek, hanem hatalmas szerepe van a közösségek kialakulásában is. Számtalan videójátékokon alapuló közösség jött létre különböző közösségi platformokon, ahol a játékosok új kapcsolatokat építhetnek ki, csapatokat alkothatnak és közösen vehetnek részt különböző eseménye-

ken. Az e-sport szerepe is jelentős mértékben megnőtt az elmúlt években: professzionális játékosok és csapatok mérkőznek meg egymással különböző bajnokságokban. Több millió nézőt vonzanak az olyan játékok, mint a „League of Legends” vagy a „Counter-Strike” és az ezekkel kapcsolatos e-sport események.

3. fejezet

A játékfejlesztés folyamata

3.1. Szoftverfejlesztési életciklus

Egy szoftver fejlesztése során, különösen egy olyan komplex folyamat esetén, mint a játékfejlesztés, nagyon fontos egy fejlesztési struktúrát követni, amiben egyértelműen meghatározhatók a fejlesztés fázisai, és ami szabályozza a projekt menetét az első tervektől kezdve egészen a karbantartási teendőkig. Egy szoftverfejlesztési életciklus meghatározza, hogy hogyan kapcsolódnak egymáshoz logikailag és kronológiailag a fejlesztési fázisok.

Egy projektnek megfelelő módszertan kiválasztása és helyes alkalmazása számtalan előnnyel jár: leegyszerűsíti, meggyorsítja a munkát, idő- és költséghatékonyságot biztosíthat, valamint alapvetően magasabb minőségű termék előállítását teszi lehetővé.

A tipikus fázisai egy szoftver életciklusának a követelmények meghatározása, a tervezés, az implementáció, a tesztelés és a karbantartás. Megkülönböztetünk például szekvenciális, iteratív, illetve inkrementális fejlesztési modelleket.

A szekvenciális módszertanok, például az egyik legrégebbi fejlesztési modell, a vízesés-modell esetén a fejlesztés fázisai egymásra épülnek, korábbi fázisokra nincs visszatérés. Ha mégis visszalépésre van szükség, az jelentős időigénnyel, költséggel és erőforrás-ráfordítással járhat. Az ilyen módszertanok szigorúan szabályozott projekt esetén lehetnek hatékonyak, egyébként ritkán használt a rugalmatlansága miatt.

Az iteratív és inkrementális fejlesztési modellek esetében a fejlesztési folyamat iterációkban történik, és minden iterációban végeznek tesztelést, fejlesztést és tesztelést. Várhatóan minden iteráció előállít egy működő prototípust vagy termékinkremenst. Ezek a módszertanok jelentősen rugalmasabbak és agilisabbak, mint a szekvenciális módszerek.

3.1.1. Követelmények meghatározása

A követelmények meghatározása a projekt által megoldandó problémát határozza meg egy dokumentum formájában. Meg kell határozni a felhasználói követelményeket, a szoftver által nyújtott funkciókat, illetve további nem-funkcionális követelményeket.

A szoftverminőség-jellemzők biztosítják egy szoftver minőségét. Ezekre épül a követelmény-specifikáció. Általános minőségjellemző a funkcionalitás, a megbízhatóság, a hatékonyság, a használhatóság, a karbantarthatóság és a hordozhatóság.

3.1.2. Tervezési fázis

Egy projektben gyakran korlátozott idő, költségvetés és erőforrás áll rendelkezésre, ezért nagyon fontos, hogy megfelelően megtervezésre kerüljön a fejlesztési folyamat. A tervezés kihagyása vagy elhanyagolása később olyan problémákhoz vezethet, amelyek megoldása jelentős időt és pénzt felemészthet.

A tervezési fázis célja, hogy lefektessük az alapokat és meghatározzunk egy jó fejlesztési struktúrát. Ebben a fázisban születik meg a projekt alapötlete, amely lehet egy narratív, egy stratégiai vagy akár egy ügyességet és reflexeket próbára tevő játék. Meghatározásra kerülnek a játék céljai, stílusa, célközönsége, valamint az alapvető játékmechanikák. Ezek azok a mechanikák, amelyek meghatározzák, hogyan működik a játék, például hogyan mozognak a karakterek, hogyan történik az interakció a világban, és milyen szabályok vonatkoznak a játékmenetre.

Fontos lépés a projekt költségvetésének és időkeretének megtervezése is. Ekkor fel kell mérni, hogy milyen erőforrások állnak rendelkezésre (például emberi erőforrások, technikai eszközök), és mennyi idő szükséges az egyes fejlesztési szakaszokhoz. A hibásan felmért költségvetés és időkorlátok akár a projekt kudarcához is vezethetnek.

A tervezési fázisban alakul ki a fejlesztői csapat is, ahol minden szerepkör egyértelműen meghatározott. Egy jól működő csapatstruktúra rendkívül fontos. Ezen felül dönteni kell a programozási nyelvről, a fejlesztő eszközökről és egyéb technikai megoldásokról. A megfelelő eszköztár kiválasztása elengedhetetlen, hiszen ez az alapja a gördülékeny fejlesztésnek.

Szükséges egy tesztelési terv kidolgozása is, amely meghatározza, hogy a fejlesztés során mikor és hogyan zajlanak a különböző típusú tesztelések. Ez biztosítja, hogy a játék minden fázisában ellenőrizhető legyen a minőség, és a hibákat még időben felismerjük és javíthassuk.

Egy játék életciklusa azonban nem ér véget a fejlesztés és tesztelés befejezésével, hiszen ezután kezdődik el a termék értékesítése, majd támogatása és karbantartása. Ezeknek a módját, idejét és helyszínét is szükséges megtervezni, ahogy az esetleges marketing eszközöket is.

Mindezek figyelembevételével nem túlzó kijelentés, hogy a tervezési fázis a fejlesztés egyik legfontosabb része, hiszen egy jól kidolgozott projekt tervvel nagyobb eséllyel lesz sikeres a projekt.

3.1.3. Pre-produkciós fázis

A tervezési fázist követően részletesen kidolgozásra kerül a játék a pre-produkciós fázisban. Ebben a szakaszban konkrét tervek és prototípusok készülnek el annak érdekében, hogy a gyártási fázisra rendelkezésre állhasson minden szükséges erőforrás és információ.

Az egyik legfontosabb feladat ebben a fázisban a dokumentáció elkészítése. A dokumentáció tartalmazza többek között a játék funkcióinak, mechanikáinak, vizuális megjelenésének vagy éppen a játékvilág felépítésének részletes leírását. Ez biztosítja a fejlesztés gördülékenységét. Minél nagyobb precizitással készül el a dokumentáció, annál egyszerűbb és gyorsabb elvégezni magát a fejlesztést. Véglegesítésre kerülnek a választott fejlesztő eszközök, technikák és programozási nyelvek. Fontos döntést hozni arról is, hogy milyen platformra fejlesztik a játékot és milyen minőségi elvárások vonatkoznak rá. Finomításra kerülnek a tesztelési tervek is.

A pre-produkció során meghatározzák a fejlesztési ütemtervet, határidőket és elkészítik a költségbecslést. Ezek reális meghatározása és betartása hozzájárul a projekt sikerességéhez.

Gyakran elkészítik ebben a fázisban a játék prototípusát, ami egy korai, leegyszerűsített verziója a játéknak, amely lehetőséget ad arra, hogy a fontosabb funkciókat kipróbálják, leteszteljék. Ez sokat segít abban, hogy még a gyártás előtt felismerjék az esetlegesen felmerülő problémákat és hiányosságokat, ami sok időt és pénzt spórolhat meg.

A pre-produkciós fázis sikeres lezárásával megkezdődhet a gyártási fázis, ahol a tényleges fejlesztés történik.

3.1.4. Gyártás

A gyártási fázisban készül el a tényleges a játék, az eddig kidolgozott tervek és elgondolások implementálásra kerülnek. Ez a fejlesztési folyamat legaktívabb és legösszetettebb része. A különböző feladatokért felelős személyek vagy csapatok általában párhuzamosan tudnak dolgozni a feladataikon.

A programozók megírják a játék kódját, elkészülnek olyan rendszerek, mint például a fizikai motor, a felhasználói felület vagy a mesterséges intelligencia. Megvalósulnak a játékmechanikák, a játékvilág és az interakciók, például a karakter irányítása.

A grafikusok elkészítik a játék vizuális elemeit, beleértve a környezetet, a karaktereket, az animációkat és a vizuális effekteket. Ebben a szakaszban nyerik el végleges formájukat a modellek, a textúrák és az animációk, amik a játék vizuális világát határozzák meg.

A játék zenéit, hanghatásait is ki kell dolgozni. Ez magában foglalja a zenei aláfestéseket, a környezeti hangokat, karakterek hangjait és más interaktív hangelemeket.

A gyártási fázis előrehaladtával a játék egyre nagyobb részei válnak játszhatóvá, elkészülnek olyan korai verziók, mint az alfa és a béta verziók. Ezek a korai verziók lehetőséget adnak arra, hogy a fejlesztők visszajelzéseket kapjanak, és ennek megfelelően kisebb-nagyobb finomhangolást végezzenek a játékon.

Amikor a játék teljesen összeállt, és már csak az utolsó simítások vannak hátra, a projekt lassan a tesztelési fázisba lép.

3.1.5. Tesztelés

A tesztelési fázisban, ahogy a neve is mutatja, a tesztelés lép előtérbe. Célja, hogy a szoftver hibamentes és stabil legyen, mielőtt publikálásra kerül. Ezt egy átfogó tesztelés biztosítja, amely alaposan vizsgálja a játék minden aspektusát.

A funkcionális tesztelés során meg kell győződni arról, hogy a játék a terveknek és a specifikációnak megfelelően működik-e. Kifejezetten azt vizsgálja, hogy a játék mechanikái, funkció helyesen valósulnak-e meg, függetlenül attól, hogy ez milyen módon történik. E tesztelés során derül fény az esetleges hibákra vagy hiányosságokra, amik miatt a játék nem elégíti ki a meghatározott követelményeket.

A nem funkcionális tesztelés vizsgálja azt, hogy mennyire jól és hatékonyan működik a szoftver. Ekkor a működés módja, nem pedig a helyessége a szempont, ellentétben a funkcionális teszteléssel. Nem funkcionális tesztelések közé tartozik a teljesítménytesztelés, a stressztesztelés, de a használhatósági és a biztonsági tesztelés is. Mindezek

biztosítják, hogy a játék megfelelő teljesítménnyel és stabilitással futhasson, kielégítő felhasználói élményt nyújtva.

A tesztelésnek kritikus szerepe van, hiszen a tesztelés alaposságán múlik, hogy milyen minőségű lesz az elkészültnek minősített szoftver. A játék sikerességét nagy mértékben befolyásolja a felhasználói élmény, amely nagyban összefügg a hibamentes működéssel. Egy játék esetében kifejezetten fontos, hogy ne merüljenek fel súlyos, a játékmenetet erősen zavaró vagy ellehetetlenítő hibák, mert az jelentősen befolyásolhatja a játék, és így a játékot készítő cég hírnevét, ami alacsonyabb bevételhez vezet rövid és hosszútávon is. Gyakori probléma, hogy egy játékot rosszul optimalizálnak, és indokolatlanul magas a rendszerkövetelménye, amit nagyon sok játékos gépe nem ér el. A játékos szempontjából ez a játék teljes használhatatlanságát jelenti.

Természetesen hibátlan szoftver nem létezik. Már egy közepesen komplex játék is több ezer különböző funkcióval rendelkezhet, amelyeket számtalan módon lehet kombinálni. Továbbá, a játékosok sokféle hardver- és szoftverkörnyezetben futtathatják a programot, a különböző operációs rendszerektől kezdve a változatos grafikus kártyákig és perifériákig. Ez a sok milliónyi lehetséges konfiguráció gyakorlatilag lehetetlenné teszi azt, hogy minden lehetséges helyzetben előre teszteljék a játékot.

3.1.6. Karbantartás

Abból kifolyólag, hogy hibátlan szoftver nem létezik, a legalaposabb tesztelések után is felmerülhetnek hibák. A játékok életciklusa a kiadás után sem ér véget, sőt, talán ekkor jelentkeznek a legnagyobb kihívások. A már publikált termékhez kell támogatást nyújtani, a felmerülő hibákat javítani, a teljesíményt optimalizálni, valamint a játékosok általi visszajelzéseket figyelembe véve módosításokat végrehajtani.

Azonban nem csak a játékélmény biztosítása és növelése az egyetlen szempont, ennél összetettebb a feladat. Egy cég szempontjából a karbantartási tevékenységek akár idő- és pénzpazarlásnak is tűnhet, hiszen a fejlesztőket fizetni kell azért, hogy javítsák a játékot, ami ráadásul jelentős időt is felemészthet. Ilyenkor a cégnek mérlegelnie kell, hogy mennyire éri meg a hibák javítása és esetleg új tartalmak bevezetése a meglévő játékokhoz, hiszen az erőforrásait akár más projektekre is fordíthatná.

Nyilvánvalóan a cég számára nem csak plusz ráfordított pénzt és időt jelent ez a fázis, hanem hosszabb távú lehetőségeket is kínál. A játékosok elégedettsége és elköteleződése hosszútávon jövedelmezőbb lehet, mint azonnal egy új projektbe kezdeni, miközben elégedetlen játékosokat hagynak maguk mögött a karbantartás költségeinek megspórolása érdekében.

3.2. A játékmotor

A videójátékok létrehozásához a játékmotorok adnak keretet. Ez egy olyan szoftverkeretrendszer, amely eszközöket biztosít a játék fejlesztéséhez, és ezáltal megkönnyíti és felgyorsítja azt. Olyan alapvető funkciókat biztosít, mint például a grafikai megjelenítés, a játékfizika, hangkezelés, mesterséges intelligencia vagy felhasználó inputjainak kezelése.

3.2.1. Unreal Engine

Az Unreal Engine az Epic Games által fejlesztett 3D-s játékmotor. Eredetileg belső nézetű lövöldözős játékokhoz fejlesztették ki az 1990-es években, viszont azóta számos más műfajban is alkalmazták. A játékmotor C++ nyelven íródott. Kimagasló grafikai képességekkel és nagyfokú testreszabhatósággal rendelkezik. Aktuális verziója, az Unreal Engine 5 2022-ben jelent meg, melynek a forráskódja elérhető GitHub-on. Ingyenesen hozzáférhető, azonban ha 1 millió amerikai dollár feletti bevételre teszünk szert a felhasználásával, abban az esetben 5%-os jogdíj illeti az Epic Gamest. Ez a díj elengedésre kerül, amennyiben a játék kizárólag az Epic Games Store-ban jelenik meg. Unreal Engine-ben készült többek között a Fornite című játék.

3.2.2. Unity

Egy másik ismert és széles körben használt játékmotor a Unity Technologies által fejlesztett Unity. Első verziója 2005-ben jelent meg. C# programozási nyelvet használ és rendkívül sokoldalú: két- és háromdimenziós játékok készítésére is alkalmas, és számos platformot támogat, mint például PC, konzolok és mobilplatformok. A mobiljátékok jelentős része a Unity játékmotorral készült. A felhasználói felülete egyszerű és intuitív, így kezdők számára különösen jó választás lehet. A Unity egy sokoldalú és könnyen hozzáférhető motor, azonban teljesítmény szempontjából hátrányban lehet az Unreal Engine-nel szemben. Unity motort használ a megjelenésekor igencsak felkapott Pokemon Go telefonos játék is.

3.2.3. Godot

A Godot egy nyílt forráskódú, ingyenes és platformfüggetlen játékmotor, ami 2D-s és 3D-s fejlesztésre is alkalmas. Teljesen ingyenes és licencdíjaktól mentés, és mivel nyílt forráskódú, szabadon módosítható. A Godot-ban fejlesztett játékok C#-ban, vagy a motor saját nyelvében, GDScriptben vannak írva. A GDScript egy magas szintű, Pythonhoz hasonló nyelv, amely lehetőséget ad a jobb optimalizációra. A motor nem csak kódolásra, hanem grafikus felületen történő kódszerkesztésre is ad lehetőséget Visual Scripting néven. Folyamatosan fejlődik, bár 3D-s képességei még nem érik el az Unreal Engine vagy a Unity szintjét. Kifejezetten népszerű az indie fejlesztők körében.

3.2.4. Egyedi játékmotorok

Egy játék fejlesztése során dönthetünk úgy, hogy nem egy meglévő játékmotort használunk, hanem egy teljesen saját keretrendszert írunk.

Ez a megközelítés számos előnyt kínál. Az egyedi motor teljes mértékben a fejlesztők kontrollja alatt áll és a projekthez alkalmazkodik. Nem tartalmaz felesleges funkciókat, kizárólag a projekt igényeit elégíti ki, ezáltal jobb optimalizációt nyújt. Könnyebben megvalósíthatók egyedi ötletek, mechanikák vagy megjelenési stílusok, hiszen nem kell aggódni a motor korlátai miatt. Továbbá az egyedi motor mentes a licencdíjaktól, ez pedig hosszú távon költségmegtakarítást jelenthet.

A számos előny mellett ugyanakkor a hátrányokkal is figyelembe kell venni. Egy játékmotor megírása rendkívül összetett, idő- és erőforrásigényes feladat, és nagyobb tapasztalatot követel meg. Egy túlságosan specifikus játékmotor esetén fennáll annak a veszélye, hogy egy új projekthez nem lesz alkalmas, így jelentős módosításokra vagy

akár teljesen új motor fejlesztésére lehet szükség. A jól ismert, bevált játékmotorok egyik nagy előnye a közösségi támogatás és a széleskörű dokumentációk elérhetősége, ami egy saját motor esetén nincs jelen. A tesztelés, a hibakeresés és a hibák javítása is kizárólag a fejlesztőcsapatot terheli egy egyedi motor esetében.

A népszerű Rockstar Games a saját RAGE motorját használja játékaihoz, mint például a Grand Theft Auto vagy a Red Dead Redemption címekhez. A CD Projekt RED, a jól ismert Witcher és Cyberpunk 2077 játékok fejlesztői pedig a REDengine nevű játékmotorjukat használják.

Összegezve az egyedi játékmotorok létrehozása komoly kihívásokkal jár, azonban ha meg tudunk birkózni ezekkel és ki tudjuk aknázni a potenciális előnyöket, akkor azok messze meghaladják a hátrányait.

4. fejezet

A szkriptelés szerepe a játékfejlesztésben

A szkriptelés lehetővé teszi, hogy a fejlesztők gyorsabban és rugalmasabban valósítsák meg a játékok egyes funkcióit. Leegyszerűsíti a komplexebb rendszereket és magasabb szintű vezérlést tesz lehetővé anélkül, hogy magában a játékmotorban kéne módosításokat végrehajtani.

4.1. Mi a szkriptelés?

A szkriptelés során olyan utasítássorozat készül, amelyet általában valamilyen szkriptnyelven írnak azzal a céllal, hogy vezérelje egy program működését vagy automatizálja a program bizonyos funkcióit.

A szkriptek magas szintű, erősen absztrakt kódok, amelyek megkönnyítik az összetettebb rendszerek vezérlését anélkül, hogy mélyebb, alacsony szintű programozást igényelnének. Az absztrakció gyakran olyan mértékű is lehet, hogy nincs is szükség kódírásra: ha például a rendszer biztosít egy grafikus felhasználói felület, amelyen keresztül egyszerűen gombnyomásokkal beállíthatók a kívánt funkciók, az is szkriptelésnek tekinthető. Szkriptelés lehet az operációs rendszer parancssorába beírt utasítások sorozata is.

Nem véletlen, hogy éppen „szkriptnek” nevezik, ami eredetileg forgatókönyvet jelent. A szkriptelés során nem bővül a program eszköztára, hanem a már meglévő eszközöket és lehetőségeket használja fel a kívánt funkciók megvalósításához. A szkript egy „forgatókönyvet” ad a programhoz: meghatározhatják, hogy mikor, mi és hogyan történjen.

A szkriptek tehát lehetővé teszik, hogy a fejlesztők gyorsan és egyszerűen alakítsák egy program egyes funkcióit, miközben az alapvető működéséhez nem nyúlnak hozzá.

4.2. A programozás és a szkriptelés összehasonlítása

Bár ránézésre nem feltétlenül látni rögtön a különbséget, jelentősen eltér egymástól a programozás és a szkriptelés. Mindkettő a szoftverfejlesztés eszköze, azonban különböző céljaik vannak.

A programozás célja a szoftverek alapvető funkcióinak létrehozása, amely mélyebb ismereteket és alaposabb tervezést követel meg. Programozás során történik a működési logika, az adat- és fájlkezelés, valamint hardverközeli funkciók megvalósítása.

Programozási nyelvek közé tartozik például a C, a C++ vagy a Java. Ezek a nyelvek általában alacsonyabb szintű hozzáférést biztosítanak a számítógép hardvereihez és erőforrásaihoz, így nagyobb szabadságot kapnak a fejlesztők.

Ezzel szemben a szkriptelés egy magasabb szintű réteget biztosít, amely nem módosítja a program alapvető működését, hanem annak bizonyos meglévő részeit vezérli. Célja a gyors és rugalmas fejlesztés. Gyakran használt szkriptnyelv a JavaScript vagy a Lua.

A programozási nyelvek szintaktikája általában bonyolultabb, mint a szkriptnyelveké, mivel a szkriptnyelvek magasabb szintű absztrakciót nyújtanak. Ez azt jelenti, hogy a szkriptnyelvek egyszerűsített, könnyebben érthető parancsokat használnak, amelyek sokkal közelebb állnak a természetes nyelvhez. Ezáltal szkriptelni jelentősen könnyebb és intuitívabb, mint egy programot megírni, amelynek a nyelve sokszor bonyolult szintaktikával, szabályokkal és konvenciókkal rendelkezhet.

A programozási nyelvek jellemzően fordított (compiled) nyelvek, míg a szkriptnyelvek inkább értelmezettek (interpreted). Ez a különbség jelentősen befolyásolja többek között a futási időt.

A fordított nyelvek esetén a megírt kódot egy fordító (compiler) gépi kódra fordítja futás előtt, ami így a processzoron már közvetlenül futtatható. A fordítási folyamat ugyan időigényes lehet, viszont ezt követően a program már gyorsabban fut, mivel nem igényel további értelmezést.

Ezzel szemben az értelmezett nyelvek egy interpreteren keresztül futnak. A fő különbség a fordítással szemben az, hogy míg a fordítás megtörténik a futtatás előtt, addig a szkriptek értelmezése és végrehajtása a futás közben, soronként történik. Ez általában lassabb futási időt eredményez, viszont nincs szükség előzetes fordításra.

4.3. A szkriptelés előnyei

A szkriptnyelvek egyszerűsége miatt a szkriptelés könnyen tanulható és alkalmazható, ami gyorsabb fejlesztést tesz lehetővé. Egy kisebb szkript megírása, módosítása és tesztelése jelentősen egyszerűbb, mint egy nagyobb programé, amely fordítást igényel. Egyszerűsége és rugalmassága miatt a szkriptelés kifejezetten ideális prototípusok gyors létrehozására.

A szkriptnyelvek egyszerű és jól olvasható szintaxisa lehetővé teszi, hogy a kezdők is gyorsan el tudják sajátítani a használatukat. Gyorsabb beilleszkedést biztosít a fejlesztői környezetbe, hozzájárul a hibák számának csökkentéséhez és megkönnyíti a kód karbantartását, de hatékonyabb munkamegosztást is lehetővé tesz.

A szkriptelés rendkívül rugalmas, testreszabható és újrahasznosítható megoldásokat tesz lehetővé. A szkriptnyelvek gyakran platformfüggetlenek, vagyis ugyanaz a szkript több különböző környezetben is futtatható minimális módosítás után. Mivel a szkriptek általában interpretált működésűek, gyorsan frissíthetők és azonnal tesztelhetők.

Jól megírt szkriptek segítségével egyszerűen létrehozhatók moduláris kódrészek, amelyet igény szerint fel lehet használni többször is, akár más projektekben is. A szkriptek alkalmasak arra, hogy összekapcsoljanak több különféle rendszert vagy alkalmazást, például végezhet műveleteket egy weboldal és egy adatbázis között.

Összességében a szkriptelés hatékony és rugalmas megoldás, amely lehetővé teszi, hogy a fejlesztők gyorsan tudjanak alkalmazkodni a változó követelményekhez és egyszerűbben tudják kezelni az összetettebb fejlesztési feladatokat.

4.4. A szkriptelés korlátai

A szkriptelés általában magas szinten történik, így nem biztosít lehetőséget memóriakezelésre és más, hardverközeli funkciók elvégzésére.

Mivel a szkript egy másik programtól függ, ami értelmezi azt, így abban az esetben, ha a program hibásan működik, a szkript sem az elvárásoknak megfelelően fog működni. Ez még nagyobb problémát jelent, ha a program forráskódját nem tudjuk sajátkezűleg javítani, akár azért, mert nincs hozzáférésünk vagy nem vagyunk elég képzettek hozzá.

Az interpretáció miatt a kód végrehajtása futás közben történik, ami hosszabb futási időket eredményezhet, szemben a fordított nyelvekkel. Ez leginkább nagy, összetett és erőforrás-igényes feladatok esetén lehet jelentős.

Szintén az interpretált módon történő futás miatt a hibák csak futásidőben jelentkeznek, ami megnehezíti a hibakeresést, illetve a hibák elhárítását.

Az interpretált működés biztonsági kockázatokat is hordozhat. Különösen ügyelni kell a megfelelő védelem kialakítására, hogy elkerülhetők legyenek bizonyos támadások, például az injekciós támadások. Ilyen például az SQL-injekció. Ezeknek a támadásoknak a lényege az, hogy valamilyen sebezhetőséget (például rosszul kezelt inputokat) kihasználva saját kódokat „injektálnak” a kódba futás közben, gyakran ártó szándékkal.

Összegezve a szkriptelésnek sok előnye van, mint például az egyszerűség, a rugalmasság és a modularitás, azonban a programozással szemben hátrányban lehet teljesítmény, erőforrásokhoz való hozzáférés, biztonság és hibakeresés szempontjából is.

4.5. Szkriptnyelvek

4.5.1. JavaScript

A JavaScript a legismertebb és legelterjedtebb szkriptnyelvek egyike, melyet azzal a céllal hoztak létre, hogy a webfejlesztésben kiegészítse a HTML és a CSS funkcionalitását. Lehetővé teszi dinamikus, interaktív tartalmak, például űrlapok vagy animációk létrehozását.

A JavaScript egy egyszerű, platformfüggetlen nyelv, szintaxisa hasonló a Java programozási nyelvhez. Rengeteg dokumentáció és oktatóanyag áll rendelkezésre, amely jelentősen megkönnyíti a tanulási és fejlesztési folyamatokat.

A nyelv webalapú játékok írására is alkalmas, például olyan keretrendszerek használatával, mint a Phaser vagy a Three.js. Emellett JavaScript alapú mobil- és asztali alkalmazások is léteznek.

4.5.2. Lua

A Lua egy könnyű és gyors, C nyelven íródott szkriptnyelv, aminek célja az volt, hogy beágyazható legyen más programokba. Támogatja az egyszerű és rugalmas kódolást, más szkriptnyelvekhez képest pedig kiemelkedő a sebessége, a bővíthetősége és a kis mérete.

Különösen vonzó játékfejlesztési szempontokból, hiszen úgy tervezték, hogy egyszerűen integrálható legyen más alkalmazásokba. Segítségével gyorsan és nagyobb erőfeszítések nélkül készíthetünk új funkciókat vagy módosíthatunk meglévőket, az alapmotor kódja módosítása nélkül.

A Lua közösségi támogatása alacsonyabb a JavaScripthez képest és korlátozottabbak a könyvtárai, így sok funkciót a fejlesztőknek kell elkészítenie, vagy külső könyvtárakat kell használnia.

4.5.3. Ruby

A Ruby egy teljesen objektumorientált, dinamikus, általalános célú nyelv. Különösen népszerű a webfejlesztésben, de a játékfejlesztés területén is alkalmazható.

A kódja egyszerű és olvasható, ugyanakkor alkalmas komplex funkciók hatékony megvalósítására is. A dinamikus adattípusok és az automatikus memóriakezelés rugalmassá teszi. Interpreterének köszönhetően platformfüggetlen.

A Ruby nyelvnek jelentősebb teljesítménybeli korlátai vannak, mint például a Luanak, ami miatt bizonyos projekteknél nem feltétlenül ideális választás. Játékfejlesztés szempontjából kevésbé elterjedt, így korlátozott támogatás és dokumentáció érhető el ezen a területen.

4.5.4. Egyedi szkriptnyelvek

Egy sajátos igényekkel rendelkező projekt esetén szükséges lehet egy egyedi szkriptnyelv fejlesztése, hogy biztosítva legyenek a megfelelő megoldások a felmerülő igényekre.

Egy egyedi szkriptnyelv testre szabható, a projekt szükségletei szerint alakítható ki. Emiatt egyszerűbb és jobb teljesítményt nyújthat, miközben kielégíti a különleges funkciók implementációjának igényeit.

A nyelv megtervezése és implementálása azonban jelentős idő- és erőforrás ráfordítást igényel, kompatibilitási problémák merülhetnek fel, valamint tanulási nehézségeket is okozhat.

Számos játékmotor kínál saját szkriptnyelvet. Például a Godot által használt GDScript lehetővé teszi a motor egyszerű és hatékony kezelését.

4.6. Szkriptelési megoldások

4.6.1. Végrehajtási mechanizmusok

Míg a játékmotorok gyakran C- vagy C-szerű nyelven íródnak, melyek a fordítás után közvetlenül a hardveren futnak natív gépi kódként, addig a szkriptek jellemzően magasabb szintű végrehajtási rétegen keresztül kerülnek futtatásra, például interpreter vagy virtuális gép segítségével.

A szkriptek futtatásának klasszikus, legegyszerűbb módja az interpretált végrehajtás, amely során a kód futás közben kerül feldolgozásra, például egy játékmotorba integrált interpreter által. Ez nagy rugalmasságot ad, hiszen a szkriptek módosítása nem igényel újrafordítást és futás közben is módosíthatók. A játékfejlesztésben ez kifejezetten előnyös a prototípus-készítés során, valamint megkönnyíti a gyors finomhangolásokat és a tesztelést. Ez azonban több számítást igényel, ami lassabb végrehajtáshoz vezet.

A modern játékfejlesztésben az interpretált végrehajtás által nyújtott teljesítmény gyakran nem kielégítő. Ezért terjedtek el a virtuális gép alapú megoldások. Ilyenkor a forráskód nem közvetlenül futás közben kerül feldolgozásra, hanem előzetesen egy úgynevezett bájtkódra fordul. A bájtkód egy platformfüggetlen, köztes utasításkészlet,

amelyet a virtuális gép értelmez és hajt végre. Mivel a virtuális gépnek már nem a bonyolult, ember által írt forráskódot kell értelmeznie, hanem egy előre feldolgozott egyszerűbb kódot, így általában jelentős teljesítményjavulás érhető el, de természetesen még mindig lassabb, mint a natív fordítás. Emellett megőrzi az interpretált végrehajtás rugalmasságát is.

A teljesítmény további növelésére alkalmas módszer a futásidejű fordítás (angolul Just-In-Time compilation, röviden JIT). Számos modern játékmotor alkalmazza ezt a modellt. Lényege, hogy a futtatókörnyezet a szkripteket (vagy azok gyakran használt, teljesítménykritikus részeit) futás közben natív kódra fordítja. Ez egy hibrid megoldás, amely ötvözi a gyors iteráció lehetőségét a natívhoz közeli futási sebességgel. Ez a módszer azonban a program indulásakor több időt vesz igénybe, továbbá megnövekedett processzor és memóriahasználat jellemzi. Biztonsági szempontból is felmerülnek megfontolandó kérdések. A JIT-nek írható és végrehajtható memóriaterületre van szüksége a kód tárolásához, ami potenciális sebezhetőséget jelenthet. Ez megsérti az úgynevezett Write XOR Execute elvet, melynek lényege, hogy egy adott memórialap vagy csak írható, vagy csak végrehajtható lehet, de sosem mindkettő egyszerre. Ennek betartása alapvető védelmet nyújt a kódinjektáláson alapuló támadásokkal szemben, ezért számos zárt platform operációs rendszere korlátozza vagy tiltja a JIT használatát.

Ahead-Of-Time (AOT) fordítás esetén már a build folyamat során megtörténik a natív kódra való fordítás, így futáskor nem történik értelmezés, bájtkód-feldolgozás és fordítás sem. Ezzel a megközelítéssel a futási teljesítmény közel megegyezik a natív kód teljesítményével, miközben kiküszöbölhetők a JIT biztonsági kockázatai és a futás közbeni megnövekedett erőforrás-igény. Fontos azonban megjegyezni, hogy az AOT használatával elveszítjük a szkriptnyelvek egyik legnagyobb előnyét, a fejlesztési rugalmasságot. Mivel minden szkriptmódosítás érvényesítéséhez újrabuildelés szükséges, a fejlesztési ciklus lelassul. Ennek ellenére az AOT fordítás széles körben elterjedt olyan platformokon, ahol a teljesítmény és a biztonság elsődleges szempont.

A legtöbb mai játékmotor hibrid megközelítést követ. A fejlesztés során ezek a motorok a gyors iteráció érdekében virtuális gépet vagy JIT fordítást alkalmaznak, míg a végleges buildhez AOT fordítást használnak a maximális teljesítmény és biztonság elérése érdekében.

4.6.2. A szkript és a játékmotor kapcsolata

A játékmotor és a szkriptek logikailag teljesen elkülönülnek, általában eltérő nyelven írt, különböző módokon fejlesztett rétegek, de szorosan integráltak. A szkriptek célja, hogy kiterjesszék a játékmotor funkcionalitását. Ehhez elengedhetetlen, hogy kommunikálni tudjanak egymással.

4.7. Szkriptelhető játékelemek

4.7.1. Játékmenet

4.7.2. HUD

4.8. Példák szkriptelést alkalmazó játékokra

4.8.1. Angry Birds

4.8.2. Don't Starve

4.8.3. World of Warcraft

4.8.4. Roblox

4.8.5. The Sims 4

4.8.6. The Elder Scrolls: Skyrim

4.8.7. Grand Theft Auto játékok

5. fejezet

Fejlesztési dokumentáció

5.1. Követelmények meghatározása

A fejlesztés elsődleges célja egy saját játékmotor létrehozása, amely integrált szkriptelési lehetőséget biztosít. A rendszernek alkalmasnak kell lennie egy egyszerűbb, háromdimenziós platformer játék futtatására, demonstrálva a szkriptnyelv és a játékmotor közötti kommunikáció hatékonyságát.

A fejlesztés fókuszában az architektúra és a szkriptrendszer áll, így nem cél a megmodernebb motorok grafikai szintjének és funkcionális gazdagságának elérése. A motor feladata, hogy biztosítsa a stabil alapműködést, a megfelelő teljesítményt és a rugalmas szkriptelhetőséget.

5.1.1. Funkcionális követelmények

Játékmotor követelmények:

- **Grafikus megjelenítés:** A motornak képesnek kell lennie háromdimenziós objektumok betöltésére, valamint textúrák megjelenítésére. Lehetővé kell tennie a játéktér bejárhatóságát és a kamera mozgatását belső nézetű (First Person View) megközelítéssel. Támogatnia kell az alapvető megvilágítási modelleket.
- **Fizika szimulálása:** A rendszernek biztosítania kell az alapvető fizikát (gravitáció, sebesség, gyorsulás) és az objektumok közötti ütközésérzékelést (Collision Detection).
- **Bemenetkezelés:** A felhasználói interakciók (billentyűzet, egér) valós idejű feldolgozása.
- **Hangkezelés:** Alapvető hangeffektek lejátszása.

Szkriptrendszer követelmények:

- **Integráció:** A választott szkriptnyelv értelmezőjének beágyazása a játékmotorba.
- **Kétirányú kommunikáció:** A szkripteknek képesnek kell lenniük meghívni a játékmotor natív függvényeit (pl. objektum mozgatása), illetve a motornak eseményeket (pl. ütközés) kell tudnia jelezni a szkriptek felé.

- **Logika szétválasztása:** A játékmenet szabályait (pl. pontgyűjtés logikája, győzelmi feltételek) a motor újrafordítása nélkül, külső szkriptfájlokban, a motor magjától elszeparálva kell megvalósítani.

A játék követelményei:

- **Irányítás:** Előre, hátra, oldalra mozgás, ugrás és guggolás.
- **Pályaszerkezet:** Statikus és dinamikus pályaelemek, platformok, amelyeken a játékos közlekedhet, valamint kikerülendő akadályok.
- **Játékmechanika:** Gyűjthető tárgyak (pl. érmék) elhelyezése a pályán, amelyek felvételekor a játékos pontot kap, és a tárgy eltűnik.
- **Felhasználói felület (HUD):** Head-Up Display megjelenítése a képernyőn, amely valós időben mutatja a játék és a játékos aktuális állapotát.
- **Menürendszer:** Főmenü biztosítása a játék indításához, valamint kilépési lehetőség.

5.1.2. Nem-funkcionális követelmények

- **Teljesítmény:** A játéknak stabil képkockasebességgel (minimum 60 FPS) kell futnia közepes hardveren is, optimalizált memóriahasználat mellett.
- **Bővíthetőség:** A rendszer architektúrájának támogatnia kell új játékelemek vagy logikák hozzáadását kizárólag szkriptírással, a motor C forráskódjának módosítása és újrafordítása nélkül.
- **Stabilitás:** A rendszernek robusztusnak kell lennie a szkriptoldali hibákkal szemben. Egy hibásan megírt szkript nem okozhatja a teljes játékmotor összeomlását.
- **Megfelelőség:** A forráskódnak követnie kell a választott programozási szabványokat. A kódolási stílusnak egységesnek kell lennie a projekt teljes egészében.
- **Biztonság:** A szkriptkörnyezetnek izoláltan kell működnie. A szkriptek nem hajthatnak végre veszélyes műveleteket az operációs rendszer szintjén (pl. tetszőleges fájlok törlése, rendszerhívások), kizárólag a motor által biztosított API-n keresztül kommunikálhatnak.
- **Karbantarthatóság:** A kódnak moduláris felépítésűnek, jól dokumentáltnak és átláthatónak kell lennie.

5.2. Tervezés

5.3. A fejlesztőkörnyezet bemutatása

5.4. A játékmotor megírása

5.4.1. Ablakkezelés

5.4.2. A színtér

5.4.3. Kamerakezelés és irányítás

5.4.4. Objektum- és textúrakezelés

5.4.5. Játékfizika

5.4.6. Ütközésérzékelés

5.4.7. Kétdimenziós grafika

5.4.8. Hangok

5.5. A szkriptelési rendszer implementálása

5.6. A játékmenet kialakítása

5.7. A felhasználói felület

5.7.1. Menü

5.7.2. HUD

5.8. Tesztelés

5.8.1. Funkcionális tesztelés

5.8.2. Nem-funkcionális tesztelés

6. fejezet

Összefoglalás

CD Használati útmutató

Ennek a címe lehet például *A mellékelt CD tartalma* vagy *Adathordozó használati útmutató* is.

Ez jellemzően csak egy fél-egy oldalas leírás. Arra szolgál, hogy ha valaki kézhez kapja a szakdolgozathoz tartozó CD-t, akkor tudja, hogy mi hol van rajta. Jellemzően elég csak felsorolni, hogy milyen jegyzékek vannak, és azokban mi található. Az elkészített programok telepítéséhez, futtatásához tartozó instrukciók kerülhetnek ide.

A CD lemezre mindenképpen rá kell tenni

- a dolgozatot egy `dolgozat.pdf` fájl formájában,
- a LaTeX forráskódját a dolgozatnak,
- az elkészített programot, fontosabb futási eredményeket (például ha kép a kimenet),
- egy útmutatót a CD használatához (ami lehet ez a fejezet külön PDF-be vagy MarkDown fájlként kimentve).